

왜 LLM이 아니면 안 되는가?

이 문서는 “어떻게 만들었는가(How)”가 아니라, “왜 이 경험은 LLM이 아니면 만들 수 없는가(Why)”에 답한다. 그리고 그 답을 말이 아니라 실측 데이터와 실제 프롬프트로 증명한다.

01 요약 — 이 게임에서 AI의 역할

이 게임에서 LLM은 NPC 대화나 텍스트 생성 같은 장식이 아니다. LLM은 게임의 **핵심 메커니즘** 그 자체다. 두 가지 일을 한다.

- **의도 추론(Intent Extraction)** — 플레이어의 임의의 자연어를 이번 전투의 정책으로 번역.
- **일반화(Doctrine Generation)** — 전투 결과를 보고 그 의도를 유닛 이름이 아니라 **역할군(원리)** 수준의 영구 교리로 승격.

이 두 가지 — 임의의 자연어에서 의도를 읽고, 그것을 규칙으로 일반화하는 것 — 은 규칙 기반 시스템으로는 불가능하다. 그것이 이 게임이 LLM을 쓰는 유일하고 충분한 이유다.

02 왜 LLM인가 — 이 표 하나

사람은 사전에 있는 말로 말하지 않는다. 아래는 **실제 측정값**이다 (Rule-based = `scripts/proof.js`, LLM = `gpt-4o` 실패율 응답).

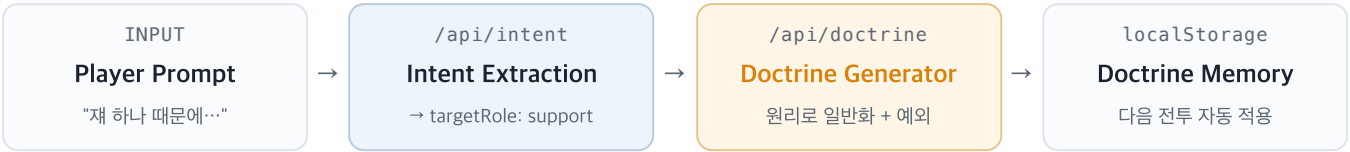
플레이어 입력	Rule-based	LLM	LLM이 읽은 의도
“적 힐러에게 화력을 집중해 먼저 처치해라”	✓	✓	회복 차단
“재 하나 때문에 안 죽잖아”	✗	✓ support	회복 차단
“뒤에 깃발 든 애 거슬러”	✗	✓ support	지원 제거
“뒤부터 죽여”	✗	✓ ranged	후방 위협 제거
“안녕하세요”	✗	✗	이해하지 못함

이 표가 동시에 증명하는 세 가지:

- **수렴** — “깃발 든 애”와 “안 죽잖아”는 공통 단어가 하나도 없는데 둘 다 support로 간다.
- **구분** — “뒤부터 죽여”는 ranged다. 아무거나 support로 뻔지 않는다.
- **절제** — “안녕하세요”는 이해하지 못한다(`understood:false`). 없는 의도를 지어내지 않는다.

특히 “재 하나 때문에 안 죽잖아” → **회복 차단**은, 규칙 엔진에 미리 정의된 것과 똑같은 결론에 GPT가 그 문장을 처음 보고 독립적으로 도달한 것이다. 정규식에서 해당 키워드는 제거되어 있다(폴백에 데모 문장을 하드코딩하지 않았다).

03 아키텍처 — 네 개의 상자



엔드포인트 / 파일	역할
/api/intent	전투 전. 자연어 → 이번 전투의 정책(targetRole, focusFire, aggression ...).
/api/doctrine	전투 후. 전투 결과 → 영구 교리(condition + action) 또는 예외.
server/proxy.js	LLM 프록시. OpenAI · Anthropic 어댑터가 동일 계약 . 키는 서버에만.
js/doctrine.js	클라이언트 심장. LLM 실패 시 규칙 기반으로 자동 강등(동일 계약).

교리는 “패턴”이 아니라 “원리”다 — 전장에 대해 평가되는 **조건부** 술어다.

```
// 교리 = 조건(condition) + 행동(action)
{
  "name": "증폭형 우선 제거",
  "condition": { "requiresTrait": "amplifier" }, // 증폭형이 없는 전장에선 발동조차 안 함
  "action":    { "targetRole": "support", "exceptTrait": "volatile" } // 지원형 우선, 단
자폭은 제외
}
```

04 실제 프롬프트 & 출력 제약

프롬프트의 핵심은 “표면의 단어가 아니라 의도를 읽어라”이고, 출력은 json_schema로 게임이 아는 값에 **강제 제약**된다.

System Prompt — Intent Extraction (발췌)

```
# /api/intent 시스템 프롬프트 (server/proxy.js)
당신은 전술 게임의 "AI 사령관"이다. 플레이어가 자연어로 내린 명령을 정책으로 번역한다.

★ 핵심: 플레이어는 게임 용어로 말하지 않는다. 일상어·비유·불평으로 말한다.
  "재 하나 때문에 안 죽잖아" → 회복시켜주는 적이 있다 → targetRole="support"
  "뒤부터 죽여" → 후방의 원거리·지원 유닛부터 → targetRole="ranged"

표면의 단어가 아니라 **의도**를 읽어라.
- targetRole은 주어진 role 목록 중 하나이거나 null. 지어내지 마라.
- 전혀 전술 명령이 아니면(예: "안녕") understood=false.
```

System Prompt — Doctrine Generation (발췌)

```
# /api/doctrine 시스템 프롬프트 (server/proxy.js)
★ 가장 중요한 원칙: 교리는 "패턴"이 아니라 "원리"여야 한다.
  ❌ "힐러를 먼저 공격한다" (유닛 이름 암기)
  ⦿ "적 전력을 증폭하는 지원형이 있으면 먼저 제거한다" (전장에 대한 술어)
- 반드시 역할군(role) 수준으로 일반화하라. (힐러도 버퍼도 support)
- 예외 학습: 교리를 적용했는데 특정 특성 때문에 졌다면, 교리를 버리지 말고
  exceptTrait에 그 특성을 넣어라. (자폭 → role="support", exceptTrait="volatile")
- 없는 의도를 지어내지 마라. 명령이 없었으면 role=null.
```

구조적 안전장치: 출력은 strict json_schema로, role·trait이 게임이 정의한 enum 값만 허용한다. → LLM은 게임 규칙을 지어낼 수 없고, 오직 의도 해석만 담당한다. 액션 스페이스는 코드가 정의하고, LLM은 그 안에서 판단한다.

05 실측 결과 (GPT-4o 실험출)

동일 프록시(OPENAI · gpt-4o)에 실제로 보낸 요청의 응답이다.

입력: "뒤에 깃발 든 애 거슬려"

```
→ { understood: true, targetRole: "support", focusFire: false,
  why: "지원 제거", rules: ["적 지원형을 최우선 제거"] }
```

입력: "재 하나 때문에 안 죽잖아"

```
→ { understood: true, targetRole: "support", focusFire: true,
  why: "회복 차단", rules: ["적 지원형을 최우선 제거", "화력 집중"] }
```

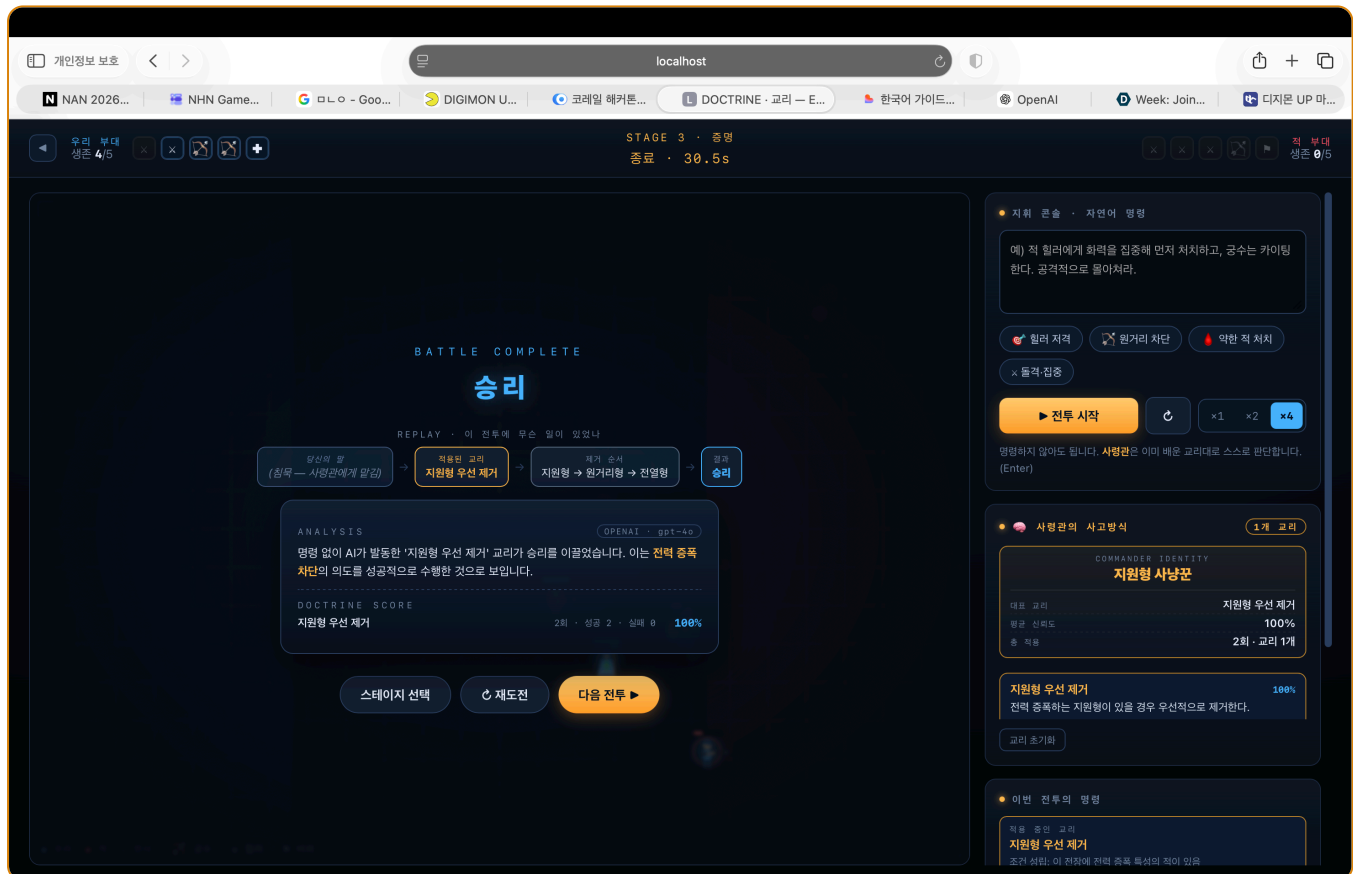
입력: "뒤부터 죽여"

```
→ { understood: true, targetRole: "ranged", focusFire: true,
  why: "후방 위협 제거", rules: ["후방 위협 제거", "화력 집중"] }
```

입력: "안녕하세요"

```
→ { understood: false, targetRole: null } // 지어내지 않는다
```

예외 학습(STAGE 4)도 GPT가 직접 판단했다. 교리를 적용해 패배한 뒤, GPT가 스스로 role="support"를 유지하면서 exceptTrait="volatile"을 출력했다 — 규칙을 버리지 않고 예외만 건 것이다.



실제 게임 화면. 플레이어가 명령하지 않았는데 학습한 교리가 발동해 승리했고, GPT-4o(OPENAI · gpt-4o 배지)가 "명령 없이 발동한 교리가 승리를 이끌었다"고 분석했다.

06 정직성 — 폴백은 LLM인 척하지 않는다

키 없이도 심사·평가가 가능하도록 **규칙 기반 폴백**을 포함한다. 그러나 규칙 기반은 정규식으로 사전 정의된 교리를 고르는 것이므로, **절대 LLM인 척하지 않는다**. Battle Complete 화면의 ANALYSIS 헤더에 추론 백엔드를 항상 표시한다.

ANALYSIS [OPENAI · gpt-4o]
실제 LLM 추론

ANALYSIS [LOCAL · 규칙 기반]
키 없는 평가 환경

두 경로는 완전히 같은 계약(ctx → {analysis, doctrine})을 지키므로, 프론트 코드는 한 줄도 바뀌지 않는다. 그리고 폴백의 한계(구어체를 이해하지 못함)는 **회귀 테스트로 고정되어 있다** — 누군가 정규식에 데모 문장을 넣으면 테스트가 깨진다.

07 개발 과정의 AI 활용 (AI-native Development)

제품의 AI(GPT-4o)와 별개로, 이 프로토타입은 Claude(Claude Code)와의 협업으로 개발되었다.

AI 도구	활용 내역
Claude Code (개발 파트너)	설계 토론 · 구현 · 리팩터링 · 밸런스 튜닝 · 회귀 테스트 작성. 자연어로 요구를 전달하면 코드로 옮기고, 헤드리스 시뮬레이터로 검증하는 루프를 반복했다.
OpenAI GPT-4o (제품 런타임)	게임 내 의도 추론 · 교리 생성. structured outputs(json_schema)로 출력을 게임 규칙에 제약.

AI 활용에서 지킨 원칙

- **검증 우선** — LLM/AI가 만든 것은 proof.js(30/30) 회귀 테스트와 헤드리스 시뮬레이션(각 30~40회)으로 반드시 검증했다.
- **연출이 아니라 사실** — “AI가 생각하는 것처럼 보이는” 가짜 연출을 거부했다. Commander 별명·교리 계보·신뢰도는 모두 실제 데이터에서 유도된 것이다.
- **정직한 폴백** — LLM이 없을 때의 규칙 기반 동작을 LLM인 척 포장하지 않았다.

재현: OPENAI_API_KEY=sk-... node server/proxy.js로 실행하면 위 모든 실측이 그대로 재현된다. 화면 배치가 OPENAI · gpt-4o로 바뀌고, 임의의 자연어 명령이 정책으로 번역된다.

08 결론

이 프로젝트에서 LLM은 텍스트를 생성하기 위해 사용되지 않았다.

LLM은 플레이어의 전략을 이해하고 일반화하기 위해 사용되었다.

그래서 LLM은 이 게임의 장식이 아니라, **핵심 메커니즘 그 자체**다. 규칙 기반 시스템은 “힐러 먼저”는 처리할 수 있어도, “재 하나 때문에 안 죽잖아”에서 의도를 읽고 그것을 힐러가 없는 전장에도 적용되는 **원리**로 일반화할 수 없다. 바로 그 지점이, 이 게임이 LLM이 아니면 존재할 수 없는 이유다.

This project does not use AI to replace the player.
It uses AI to remember, generalize, and evolve the player's strategy.